Metro Navigator & GPS Logger — Technical Design Document

Detailed design, algorithms, configuration, edge cases, testing and operations

24 September 2026

Document control

1. Repository layout
2. Build, run and test
3. Module `:core`
   - 3.1 `GpsPoint`
   - 3.2 `GeoToolbox`
   - 3.3 `NetworkStatus`
   - 3.4 `TelemetryLogger`
4. Module `:app` — Metro Navigator
   - 4.1 Runtime structure
   - 4.2 `SensorHub`
   - 4.3 `MetroRoute` and `TrackStitcher`
   - 4.4 `MetroRouteService`
   - 4.5 `UndergroundNavigator` (positioning engine)
   - 4.6 `NavigationController`
   - 4.7 `LocationTrackerService`
   - 4.8 `MainActivity`
5. Module `:gpslogger` — GPS Logger
6. Configuration reference (all tunable constants)
   - 6.1 Navigator (`UndergroundNavigator.kt`)
   - 6.2 Sensors (`SensorHub.kt`)
   - 6.3 Data and orchestration
7. Threading and lifecycle
8. Error handling
9. Permissions (manifest)
10. Edge-case catalogue
11. Accuracy results (simulation)
12. Testing
    - 12.1 Automated tests (39, all passing)
    - 12.2 Static analysis
    - 12.3 On-device checks performed (Samsung Galaxy S24+, Android 14)
13. Field validation procedure (required before production)
    - 13.1 Tuning guide
14. Known limitations and future work
15. Change history (from the original project)
16. Troubleshooting

Document control

Item	Value
Document	Technical (detailed / low-level) design
Version	1.0 — 24 September 2026
Applies to	`:core`, `:app` (Metro Navigator), `:gpslogger` (GPS Logger)
Companion	`01_High_Level_Design.md`
Status	Code complete · 39 unit/simulation tests passing · Android Lint 0 errors · on-device smoke tested (Samsung Galaxy S24+, Android 14) · **field ride validation pending**

1. Repository layout

```
TestGPSLocation/
├── settings.gradle.kts            modules: :app, :core, :gpslogger
├── build.gradle.kts               AGP 8.5.0, Kotlin 1.9.0 plugins
├── gradle/wrapper/                Gradle 8.8
├── core/                          Android library (shared)
│   └── src/main/kotlin/com/example/testgpslocation/
│       ├── models/Models.kt               GpsPoint
│       └── utils/
│           ├── GeoToolbox.kt              distance, bearing, destination point
│           ├── NetworkStatus.kt           NONE / WEAK / GOOD
│           └── TelemetryLogger.kt         daily log files, async writer
├── app/                           Metro Navigator (com.example.testgpslocation)
│   └── src/main/java/com/example/testgpslocation/
│       ├── MainActivity.kt                UI, warnings, permissions
│       ├── sensor/SensorHub.kt            GNSS, network, IMU, barometer, steps
│       ├── engine/
│       │   ├── UndergroundNavigator.kt    positioning engine (pure Kotlin)
│       │   ├── MetroRoute.kt              line geometry + TrackStitcher
│       │   ├── MetroRouteService.kt       Overpass download + offline cache
│       │   └── NavigationController.kt    orchestration singleton
│       └── service/LocationTrackerService.kt  foreground service + logging
│   └── src/test/.../engine/               39 JVM tests (see §14)
├── gpslogger/                     GPS Logger (com.example.gpslogger)
│   └── src/main/java/com/example/gpslogger/
│       ├── MainActivity.kt, LoggerService.kt, LocationLog.kt
└── docs/                          this documentation (MD, DOCX, PDF)
```

2. Build, run and test

Requirements: Android Studio (Koala or newer) or command-line Android SDK with platform 34; JDK 17+ for Gradle; a device/emulator with Android 7.0+ (a real phone is needed for sensors).

Task	Command
Build both apps (debug)	`./gradlew assembleDebug`
Run all unit + simulation tests	`./gradlew testDebugUnitTest`

Task	Command
Lint	`./gradlew lintDebug` (reports in `*/build/reports/lint-results-debug.*`)
Install Metro Navigator	`adb install -r app/build/outputs/apk/debug/app-debug.apk`
Install GPS Logger	`adb install -r gpslogger/build/outputs/apk/debug/gpslogger-debug.apk`
See simulation metrics	`./gradlew :app:testDebugUnitTest -i | grep SIM`
Pull today's logs	`adb pull /sdcard/Download/TestGPSLocation/`

Release: add a `signingConfigs.release` block, then `./gradlew assembleRelease` or `bundleRelease`. If R8 minification is enabled, no special keep rules are required (no reflection; `org.json` and Play services ship their own rules).

3. Module `:core`

3.1 `GpsPoint`

Field	Meaning
`latitude`, `longitude`	WGS-84 degrees
`accuracy`	Horizontal accuracy radius (m, 68 %)
`timestamp`	UTC wall clock (ms) — display/logs only
`elapsedRealtimeMs`	Monotonic time of the fix — **all maths uses this** (immune to clock/time-zone changes)
`speedMeterPerSecond`, `hasSpeed`	Doppler speed if provided
`bearingDegree`, `hasBearing`	Course over ground if provided
`isGnss`	`true` = satellite receiver fix; `false` = network/fused
`isMock`	Fix came from a mock-location provider

3.2 `GeoToolbox`

- `calculateDistance` — haversine, Earth radius 6 371 000 m.
- `calculateBearing` — initial great-circle bearing, 0–360°.
- `bearingDifference` — smallest absolute angle between two bearings (0–180°).
- `moveLocation(lat, lon, d, bearing)` — destination point on the sphere.

3.3 `NetworkStatus`

`quality(context)` returns:

- **NONE** — no active network or no `INTERNET` capability (or permission error).
- **WEAK** — not `VALIDATED` (captive portal / no real internet), estimated downstream bandwidth < 250 kbps, or cellular signal < –110 dBm (Android 10+).
- **GOOD** — otherwise.

`label()` → “Lost” / “Weak” / “Connected” (used in logs and UI). Requires `ACCESS_NETWORK_STATE`, declared in the core manifest so every app inherits it.

3.4 `TelemetryLogger`

- **File name:** `<AppLabel>_telemetry_<yyyy-MM-dd>.txt` (label sanitised to `[A-Za-z0-9_.-]`). A new file starts automatically at midnight.
- **Location preference:**
  1. Android 10+: `MediaStore.Downloads`, relative path `Download/TestGPSLocation/` (no permission needed; the entry is looked up by name + path, else inserted).
  2. Android ≤ 9: public `Download/TestGPSLocation/` if `WRITE_EXTERNAL_STORAGE` is granted.
  3. Otherwise / on any write failure: app-specific `Android/data/<pkg>/files/Download/`.
- **Threading:** a single background thread executes writes in call order; callers never block.
- **Failure handling:** a failed write falls back to app storage and retries once; the last error text is kept in `TelemetryLogger.lastError` and shown by the UI (e.g. storage full).
- **API:** `log(...)` (position line, optional `extra` columns), `logCustom(msg)`, `appendLine(line)`, `clearLogs()`, `readRecentLines(n)`, `getLogLocation()`.

Position line format (Metro Navigator):

```
[2026-09-24 19:31:02.114] Lat: 12.910401, Lon: 77.580955 | Source: TRACK | Speed: 17.84 m/s
  | GPS: Lost | Network: Lost | Error: ±142 m | Satellites used: 0 | Line: Green Line
  (Madavara -> Silk Institute) | Next: Jayanagar
```

(single line in the file; wrapped here). Event line: `[time] EVENT: Stopped at Jayanagar`.

4. Module `:app` — Metro Navigator

4.1 Runtime structure

```
MainActivity ──acquire("activity")──┐
                                    ├──> NavigationController (object, main thread)
LocationTrackerService ─acquire("service")┘      │
                                                 ├─ SensorHub (listener = controller)
                                                 ├─ UndergroundNavigator
                                                 └─ MetroRouteService
```

Sensors run while at least one client holds the controller: the visible screen and/or the logging service. Therefore navigation continues in the background only while logging is on.

4.2 `SensorHub`

Location sources

Source	API	Interval	Marked as
Satellite receiver	`LocationManager.GPS_PROVIDER`	1 s	`isGnss = true`
Google fused	`FusedLocationProviderClient`, HIGH_ACCURACY (or BALANCED if only approximate permission)	2 s (min 1 s)	`isGnss = false` (except: phone without GNSS hardware and accuracy ≤ 25 m)

Source	API	Interval	Marked as
Platform network (no Play services, or fused failure)	`LocationManager.NETWORK_PROVIDER`	2 s	`isGnss = false`
Satellite status	`GnssStatus.Callback`	on change	satellites in view / used in fix

Fixes older than 30 s (by `elapsedRealtimeNanos`) are dropped. Mock flag from `Location.isMock` (Android 12+) or `isFromMockProvider`.

GPS-lost watchdog: every second; if no satellite fix for **6 s** (measured from the last fix, or from start-up) → `onGpsLost` once, until the next satellite fix.

Motion pipeline (every 100 ms):

1. Linear acceleration at `SENSOR_DELAY_GAME` (~50 Hz). Fallback when absent: raw accelerometer minus a low-pass gravity estimate (α = 0.9).
2. Rotation vector gives the device→world matrix *R*; world acceleration `east = R0·x + R1·y + R2·z`, `north = R3·x + R4·y + R5·z`; averaged over the window.
3. Classification from the last ~2 s (100 samples):
   - **WALKING** — ≥ 3 step-detector events in 3 s, or acceleration-magnitude std > 1.2 m/s².
   - **STATIONARY** — std < 0.06 m/s² (0.09 for raw accelerometer) **and** gyroscope mean < 0.05 rad/s, continuously for ≥ 1 s. `stationaryForMs` counts how long.
   - **MOVING** — otherwise, and also while fewer than 10 samples exist (warm-up; see §12 E-31).
4. Barometer: pressure change > 0.4 hPa within 10 s → `pressureRapidChange` (stairs/escalator).
5. `dtSec` is capped at 1 s so a pause (e.g. device doze) never integrates a huge step.

`DeviceCapabilities` reports GNSS, Play services, linear acceleration, accelerometer, rotation vector, gyroscope, barometer and step detector presence for the warning banner.

4.3 `MetroRoute` and `TrackStitcher`

MetroRoute (immutable): `id`, `lineName`, `fromStation`, `toStation`, `points` (polyline), `stations` (with `distanceAlongTrack`), cumulative distances, `length`.

- **Projection** `project(lat, lon)` → (distance along track, perpendicular offset). For each segment a local equirectangular frame is used (1° lat = 111 320 m; 1° lon = 111 320·cos φ); the point is projected with clamped parameter $t \in [0,1]$; the smallest offset wins. Along-track distance = cumulative[i] + t·segmentLength (haversine segment lengths).
- **pointAt(d)** — binary search on cumulative distances + linear interpolation.
- **bearingAt(d, direction)** — segment bearing; +180° for direction −1.
- **Stations:** projected; dropped if > 400 m from the track; sorted by along-track distance; de-duplicated by name or when closer than 60 m to another station.
- **nextStation(d, dir, tolerance = 3 m)**, **previousStation**, **nearestStation**.

TrackStitcher.stitch(ways) joins OSM way geometries into one polyline: start with the first way; repeatedly attach the unused way whose end point is nearest to either end of the chain (reversing it if needed); stop when the best gap exceeds 50 m (so disconnected pieces such as a depot spur are not joined by a straight jump). Duplicate joint points are removed.

4.4 `MetroRouteService`

Overpass query (normal, 2 km radius; timeout 60 s):

```
[out:json][timeout:60];
rel["route"~"^(subway|light_rail|monorail)$"](around:2000,LAT,LON)->.r;
.r out geom;                        // relations with inline member geometry
node(r.r)["name"];out body;         // named stop/station nodes of those relations
way["railway"~"^(subway|light_rail|monorail)$"](around:2000,LAT,LON);out geom;
node["railway"~"^(station|halt)$"](around:2000,LAT,LON);out body;
```

The **whole-city** download (30 km) uses only the first three statements, timeout 180 s.

Design note: an earlier version recursed into every track node (`((._;>);); out body geom`), which produced very large responses and HTTP 504 timeouts. The inline-geometry query returns all Bengaluru lines around the test location in ~8 s (~260 KB).

Servers (tried in order until one succeeds): overpass-api.de, overpass.kumi.systems, overpass.private.coffee. Connect timeout 10 s, read timeout 70 s (190 s city). A 200 response containing an Overpass `runtime error` remark is treated as a failure.

Parsing rules

- A relation is metro if `route` ∈ {subway, light_rail, monorail}; a way if `railway` ∈ same.
- Relation way members with role starting `platform` are ignored (platforms are not track).
- Stations: member nodes with role `stop*`, or nodes tagged `railway=station|halt|stop` or `public_transport=stop_position`; name from `name:en`, else `name`.
- If a relation has no stop nodes, all station nodes in the response are offered (filtered by distance to the track).
- No relations at all → loose metro ways are stitched into one line.
- Line name: `name:en` → `name` → `Line <ref>` → "Metro line". `from`/`to` tags kept.
- Null / NaN coordinates are skipped (clipped geometry can contain nulls).

Offline cache: `files/metro_routes/<rel_id>.json` (version 2) written to a `.tmp` file then renamed (atomic; no half-written lines after a crash or full disk). Corrupt or old-version files are deleted on load; leftover `.tmp` files are removed.

```
{"version":2,"id":"rel_7842287","lineName":"Green Line (Madavara -> Silk Institute)",
 "from":"Madavara","to":"Silk Institute",
 "track":[[13.0578,77.4722],[13.0571,77.4730], ...],
 "stations":[{"name":"Madavara","lat":13.0576,"lon":77.4724}, ...]}
```

Downloaded areas: `downloaded_areas.json` = list of {lat, lon, radius, time}. A position is "covered" if within (radius − 500 m) of an area downloaded in the last 7 days → no network call.

Concurrency: `cacheMutex` guards the in-memory list and files; `downloadMutex` serialises downloads. Offline lookups (`cachedNear`) take only `cacheMutex` and therefore never wait for a slow download.

Results: `Saved(n)`, `AlreadyDownloaded`, `NothingHere`, `NoNetwork`, `Failed(reason)`.

4.5 `UndergroundNavigator` (positioning engine)

4.5.1 State

Position (lat, lon, source, uncertainty σ), satellite state (available, last fix time, loss time, reject streak, recovery candidate), motion (speed, cruise speed, heading, motion state), track state (route, on-track flag, along-track distance *s*, direction ±1, direction-known, destination, held station, overshoot, snapped-at-station), departure-detection accumulators, statistics (distance since loss, recovery error, station stops, network corrections).

4.5.2 Location input

```
onLocationFix(fix):
  reject if accuracy ≤ 0 or > 2000 m, NaN, out of range, or exactly (0,0)
  if fix.isGnss and accuracy ≤ 50 m  → handleGnss
  else                               → handleWeak
```

handleGnss

1. *Jump filter:* implausible if (distance − acc_i − acc_j)/Δt > 60 m/s. Rejected, unless it is the 3rd consecutive rejection → accept (our reference was wrong).
2. *Recovery confirmation* (first fix after an outage): trusted immediately only if accuracy ≤ 15 m; otherwise it is used as a weak fix and must be confirmed by a second fix within 40 m + max(speed, 5 m/s)·Δt. On confirmation, the **recovery error** (distance between the estimate and the fix) is recorded and logged with the predicted ±σ.
3. Speed from Doppler; cruise speed learnt: `cruise = 0.9·cruise + 0.1·speed` when speed > 4 m/s (bounded 5–30 m/s).
4. Stationary jitter filter: speed < 0.5 m/s and move < 15 m → keep previous coordinates.
5. σ = fix accuracy; source = GPS; heading from bearing if speed > 1.5 m/s.
6. Track: on-track if offset ≤ 60 m + accuracy; *s* = projection; direction from GPS bearing (speed > 2 m/s: forward if |bearing − trackBearing| ≤ 90°) or from ≥ 20 m of along-track progress.

handleWeak (network fixes; poor/unconfirmed satellite fixes) — ignored while GPS is good. Measurement noise R = max(accuracy, 10 m) × 1.5.

- *No position yet:* adopt it (source NET), lock to a line if within min(acc + 50, 250) m.
- *Line known, not on track:* lock if within min(acc + 50, 250) m.
- *On track:* skip if the train is standing at a snapped station; skip if the fix is more than acc + 150 m off the line; innovation v = s_meas − s; reject if |v| > 3·√(σ² + R²); otherwise Kalman update `k = σ²/(σ²+R²)`, `s += k·v`, `σ = √(σ²R²/(σ²+R²))`.
- *No line:* same gating and blend applied to latitude/longitude.

4.5.3 GPS lost

1. Ignored if already lost. Records loss time; resets per-trip counters.
2. **Cold start** (never had a satellite fix and no line): nothing to carry forward → return.
3. Lock to a line if the position is within 250 m of it.
4. On track:
    - speed < 2 m/s (standing, e.g. boarding): snap to a station within 250 m, remember it as the last stop, start watching for the departure.
    - otherwise: advance along the track by `speed × gap` where gap = time since the last fix (≤ 15 s) — compensates the 6 s detection delay (e.g. 120 m at 20 m/s).
    - σ = max(σ, 20 m) + gap·(0.15·speed + 0.3).

5. Off track: dead-reckon the gap along the last heading; source DR.

4.5.4 Motion input (every 100 ms, only without GPS)

Motion state	On a line	Off a line
STATIONARY	speed = 0; restart departure window; after 5 s still → **station stop handling**	speed = 0
WALKING	train speed = 0 (walking on platform/carriage does not move the train)	1.2 m/s along heading
MOVING	`speed += a_along·dt` (clamped ±2.5 m/s²), then `speed += (cruise − speed)·dt/τ` with τ = 20 s (direction known) or 6 s (unknown)	speed decays from last GPS speed with τ = 60 s — **no invented motion**

`a_along = east·sin(b) + north·cos(b)` where *b* = track bearing in the travel direction. Speed is clamped to 0–40 m/s. Step = speed·dt.

Uncertainty growth per step: on a line `σ += dt·(0.15·speed + 0.3)`; off a line `σ += dt·(0.3·speed + 1.0)`.

4.5.5 Advancing along the track (station hold)

```
if direction unknown and watching departure and no destination:
    accumulate step in undirectedDistance (applied once direction is known); return
if holding at station H:
    overshoot += step
    if overshoot > allowance: log "Passed H without stopping"; s = H + dir·overshoot; release
    return
next = first station strictly ahead
if s + dir·step passes next:
    hold at next − dir·5 m; allowance = max(150 m, 0.5 × segment length)
    (segment = station strictly behind next → next)
else s += dir·step
```

Rationale: the speed model is approximate; holding the estimate at the next platform until the train actually stops keeps station names correct, while the allowance still lets the estimate move on when a train skips a station.

4.5.6 Station stop handling

After 5 s continuously still on a line:

- Candidates: the next station if within max(400 m, 0.6 × segment) + overshoot ahead; the previous station if within 250 m behind. The nearest candidate wins; none → “Train stopped between stations (signal?)”.
- Snap *s* to the station; σ = min(σ, 40 m); mark snapped-at-station; count the stop.
- **Cruise learning:** if the previous stop is known and the moving time was ≥ 20 s, observed = segment length / moving time; if 5–30 m/s, `cruise = 0.5·cruise + 0.5·observed`.
- Log “Stopped at X” (and “Arrived at destination X”).

4.5.7 Departure detection and interchange

The departure window starts at every still moment. During the first 4 s of MOVING the world acceleration is averaged. If its magnitude ≥ 0.25 m/s², its bearing is the direction of travel:

1. **Interchange:** among the current line and other stored lines within 300 m, pick the one whose track bearing (either direction) best matches; switch only if it is ≥ 20° better and is a different line (not just the other-direction relation of the same line).
2. **Direction:** forward if |departureBearing − trackBearing| ≤ 90°, else backward (unless the user chose a destination). Logged as "Departure detected, heading towards X".
3. Distance accumulated while the direction was unknown is applied in the decided direction.

4.5.8 Outputs (`NavState`)

Latitude/longitude, source (GPS/NET/TRACK/DR), GPS available, underground (no GPS ≥ 5 s or rapid pressure change), speed, heading, motion, time/distance since loss, **uncertainty ±m**, GPS accuracy, confidence (HIGH ≤ 30 m, MEDIUM ≤ 100 m, LOW ≤ 300 m, VERY_LOW), status text, line, on-track, direction-known, previous/next station, distance and ETA to next (`distance / max(speed, 0.6·cruise)`), destination (chosen, else line terminus in travel direction), stops remaining, station list, recovery error, stops detected, network corrections, cruise speed, mock flag.

4.6 `NavigationController`

- `init(context)`: creates SensorHub and MetroRouteService, preloads the cache (line count shown immediately), registers a default-network callback.
- `acquire(client)` / `release(client)`: sensors start with the first client, stop after the last.
- **Offline lookup** (`maybeLookupLines`): at most every 10 s or 200 m (forced on GPS loss and on network return). Gets stored lines within 400 m (nearest first), sets alternatives, switches the active line only if the current one is > 400 m away, updates line status, and schedules a download if the area is not covered.
- **Download scheduling:** back-off 30 s → 60 → 120 … ≤ 10 min after failures; reset on success; "no network" does not grow the back-off and the network callback retries immediately. User actions: *Refresh lines here* (force, 2 km), *Download whole city* (force, 30 km).
- **Outputs:** `state` (≤ 4 Hz, immediate on fixes/events), `history` (500 lines; GPS lines only when moved ≥ 5 m or every 10 s), `positions` (every GPS fix ≥ 1 s apart; every 2 s without GPS), `messages` (events), `cachedLines`, `lineStatus`, `satellites`.
- Coroutine scope on the main dispatcher with `SupervisorJob` and an exception handler (a failed background task is logged, never crashes the app).

4.7 `LocationTrackerService`

- Foreground service, type `location` (`startForeground(..., FOREGROUND_SERVICE_TYPE_LOCATION)` on Android 10+). If the start is refused (permission revoked, background-start restriction) the service stops itself cleanly.
- Acquires the controller; **partial wake lock** (cap 12 h) so the CPU keeps delivering motion sensor events with the screen off.
- Writes every `PositionEvent` with extra columns (error, GPS accuracy, satellites, line, next station, MOCK flag) and every event message; notification text shows status + next station, updated at most every 3 s.
- `START_STICKY`; `isRunning` flow drives the Start/Stop buttons.

4.8 `MainActivity`

- Starts the controller on `onStart`, releases on `onStop`.

- Shows: status (GPS ACTIVE / NO GPS · UNDERGROUND NAV / NETWORK / DEAD RECKONING), network quality, coordinates, source, GPS accuracy, satellites used/in view, motion state, last recovery error, line, previous → next station, distance and ETA, destination and stops, speed km/h, heading, distance since GPS loss, **±error** (colour by confidence), line-data status, history (colour: GPS green, NET purple, TRACK/DR blue, events orange).
- Controls: destination spinner, *Flip*, *Refresh lines here*, *Download whole city*, *Start/Stop Logging*.
- **Warning banner** (highest priority first) with a *Fix* action:

#	Condition	Fix action
1	No location permission	Request, or open app settings if permanently denied
2	Only approximate location	Open app settings
3	Location services off	Google “turn on location” dialog, else system settings
4	Log write error	— (message shows the reason)
5	Logging and battery optimisation active	Open battery optimisation settings
6	Notifications disabled (Android 13+)	Open app notification settings
7	No motion sensors	—
8	No GNSS receiver	—
9	No rotation-vector sensor	— (advice: choose destination)
10	Physical-activity permission missing	Request
11	Mock location detected	—

5. Module `:gpslogger` — GPS Logger

- `LoggerService`: foreground service (type location); Fused HIGH_ACCURACY every 2 s (min 1 s); each fix (except invalid 0,0) → `LocationLog.add`.
- `LocationLog`: formats `yyyy-MM-dd HH:mm:ss, lat, lon, ±acc m`, appends to the file via `TelemetryLogger.appendLine`, keeps the newest 1000 lines in a `StateFlow`; on first screen open after a restart, reloads today’s file (duplicates removed); `clear()` truncates the file.
- `MainActivity`: current fix, status (Logging/Stopped · N points), file location, Start/Stop/Clear, list (auto-scroll when at top), “Location is turned off — tap to enable” banner, log write error display, permission requests.

File example:

```
2026-09-24 19:40:12, 12.910401, 77.580955, ±4 m
```

6. Configuration reference (all tunable constants)

6.1 Navigator (`UndergroundNavigator.kt`)

Constant	Value	Purpose
DEFAULT_CRUISE_MPS	12.0	Initial cruise speed (43 km/h) before learning

Constant	Value	Purpose
MIN / MAX_CRUISE_MPS	5.0 / 30.0	Bounds of learnt cruise speed
CRUISE_LEARN_MIN_MPS	4.0	GPS speed above which cruise is learnt
MAX_SPEED_MPS	40.0	Speed clamp (144 km/h)
MAX_PLAUSIBLE_SPEED_MPS	60.0	GPS jump filter
MAX_ACCEL_MPS2	2.5	IMU acceleration clamp
WALKING_SPEED_MPS	1.2	Walking speed off a line
CRUISE_PULL_TAU_SEC / _UNCERTAIN_SEC	20 / 6	Speed pull time constants
FREE_SPEED_DECAY_TAU_SEC	60	Speed fade off a line
MAX_USABLE_ACCURACY_METERS	2000	Fixes worse than this are ignored
GNSS_GOOD_ACCURACY_METERS	50	Satellite fixes above this are “weak”
RECOVERY_INSTANT_ACCURACY_METERS	15	Trust immediately after an outage
RECOVERY_CONFIRM_METERS	40	Consistency radius for confirmation
REJECT_RESET_COUNT	3	Consecutive jumps before accepting
WEAK_ACCURACY_INFLATION	1.5	Network fix noise multiplier
WEAK_GATE_SIGMAS	3.0	Outlier gate
WEAK_OFF_TRACK_TOLERANCE_METERS	150	Max network offset from line (+accuracy)
STATIONARY_DRIFT_METERS	15	GPS jitter filter while standing
MAX_GAP_SEC	15	Max loss-detection gap dead-reckoned
LOCK_RADIUS_METERS	60	Track lock with GPS (+accuracy)
LOST_LOCK_RADIUS_METERS	250	Track lock without GPS
STATION_ENTRY_RADIUS_METERS	250	Snap to station when GPS lost while standing
INTERCHANGE_RADIUS_METERS	300	Candidate lines at departure
LINE_SWITCH_MARGIN_DEG	20	Required bearing improvement to switch line
STATION_STOP_MS	5000	Stillness needed for a station stop
STOP_SNAP_AHEAD_MIN_METERS / _FRACTION	400 / 0.6	Snap window ahead
STOP_SNAP_BEHIND_METERS	250	Snap window behind
MIN_OVERSHOOT_METERS / OVERSHOOT_FRACTION	150 / 0.5	Station-skip allowance
HOLD_MARGIN_METERS	5	Hold point before the station
DESTINATION_MIN_DISTANCE_METERS	30	Destination sets direction beyond this
MIN_SEGMENT_SECONDS_FOR_LEARNING	20	Minimum segment time for cruise learning

Constant	Value	Purpose
DEPARTURE_WINDOW_SEC / DEPARTURE_MIN_ACCEL	4 / 0.25	Departure detection
MIN_TRACK_UNCERTAINTY_METERS	20	σ floor at GPS loss
STATION_SNAP_UNCERTAINTY_METERS	40	σ after a station snap
TRACK_SPEED_ERROR_FRACTION / TRACK_BASE_ERROR_MPS	0.15 / 0.3	σ growth on a line
FREE_SPEED_ERROR_FRACTION / FREE_BASE_ERROR_MPS	0.3 / 1.0	σ growth off a line
UNDERGROUND_AFTER_MS	5000	“Underground” flag delay

6.2 Sensors (`SensorHub.kt`)

Constant	Value	Purpose
SAMPLE_PERIOD_MS	100	Motion sample period
GPS_TIMEOUT_MS	6000	“GPS lost” after this silence
GNSS_INTERVAL_MS / FUSED_INTERVAL_MS	1000 / 2000	Location update intervals
STALE_FIX_MS	30000	Drop older fixes
NO_GNSS_TRUSTED_ACCURACY	25	Fused fix treated as satellite on phones without GNSS
ROLLING_SAMPLES / MIN_SAMPLES_FOR_STD	100 / 10	Classification window / warm-up
STEP_WINDOW_MS / MIN_STEPS_FOR_WALKING	3000 / 3	Walking by steps
WALKING_ACCEL_STD	1.2	Walking by vibration
STILL_ACCEL_STD / _RAW	0.06 / 0.09	Stillness threshold
STILL_GYRO_MEAN	0.05	Stillness (rotation)
STILL_CONFIRM_MS	1000	Stillness confirmation
GRAVITY_ALPHA	0.9	Gravity low-pass (fallback)
PRESSURE_WINDOW_MS / PRESSURE_RAPID_DELTA_HPA	10000 / 0.4	Rapid pressure change

6.3 Data and orchestration

Constant	Value	Where
SEARCH_RADIUS_METERS	2000	Normal download radius
CITY_RADIUS_METERS	30000	Whole-city download
MAX_ROUTE_DISTANCE_METERS	400	“Near a line”
AREA_MAX_AGE_MS	7 days	Downloaded-area memory
AREA_EDGE_MARGIN_METERS	500	Area coverage margin

Constant	Value	Where
CONNECT / READ / CITY_READ timeouts	10 / 70 / 190 s	HTTP
MAX_STATION_OFFSET_METERS	400	Station must be this close to track
MIN_STATION_SPACING_METERS	60	Station de-duplication
ON_STATION_TOLERANCE_METERS	3	“At station” tolerance
LOOKUP_INTERVAL_MS / LOOKUP_DISTANCE_METERS	10 s / 200 m	Offline lookup throttle
INITIAL / MAX_BACKOFF_MS	30 s / 10 min	Download retry
MAX_HISTORY	500	On-screen history
MAX_WAKE_LOCK_MS	12 h	Wake-lock safety cap

7. Threading and lifecycle

Work	Thread
Sensor events, location callbacks, navigator, controller state	Main thread (Handler / main Looper) — the navigator is single-threaded by design
Overpass HTTP, JSON parsing, cache file I/O	`Dispatchers.IO`
Offline projection lookups	`Dispatchers.Default`
Log writes	Single-thread executor (`telemetry-writer`)
UI collection	`lifecycleScope` + `repeatOnLifecycle(STARTED)`

Process death: the navigator state is in memory; after a restart it re-acquires GPS and the stored lines load from disk. The on-screen history of Metro Navigator restarts empty (the log file keeps everything); GPS Logger reloads its list from today’s file.

8. Error handling

Failure	Handling
Location permission missing / revoked	Sensors not started; warning with fix; service stops itself
`startForeground` refused	Caught; service stops; UI shows Start enabled
Play services missing or fused request fails	Platform network provider used
No GNSS hardware	Fused fixes ≤ 25 m treated as satellite; warning
Overpass HTTP error / timeout / runtime error	Next server; then `Failed(reason)`; back-off
Corrupt JSON from server	`Failed("bad data…")`; nothing written
Corrupt cache file	Deleted on load
Crash during cache write	Atomic temp-file rename; leftovers deleted
Log write failure / storage full	Fallback to app storage; `lastError` shown
Sensor sample with NaN / invalid rotation vector	Sample skipped
Background coroutine exception	Caught by handler, logged

9. Permissions (manifest)

Metro Navigator: `ACCESS_FINE_LOCATION`, `ACCESS_COARSE_LOCATION`, `ACTIVITY_RECOGNITION`, `INTERNET`, `ACCESS_NETWORK_STATE`, `FOREGROUND_SERVICE`, `FOREGROUND_SERVICE_LOCATION`, `POST_NOTIFICATIONS`, `WAKE_LOCK`, `WRITE_EXTERNAL_STORAGE` (maxSdk 28). GPS Logger: the location, foreground-service, notification and legacy storage permissions, plus `ACCESS_NETWORK_STATE` inherited from `:core`. Background location (`ACCESS_BACKGROUND_LOCATION`) is **not** needed: tracking in the background happens only inside a foreground service started while the app is visible.

10. Edge-case catalogue

ID	Situation	Behaviour	Where	Test
E-01	GPS good, network good	GPS position; lines auto-download	Navigator, Controller	device smoke test
E-02	GPS good, no network	GPS position; stored lines; retry on reconnect	Controller	—
E-03	GPS good, weak network	Downloads back off; offline lookup unaffected	Service/Controller	—
E-04	Network fix while GPS good	Ignored	Navigator	`networkFixIsIgnoredWhileGpsIsGood`
E-05	Train enters tunnel at speed	Gap dead-reckoned; track navigation	Navigator	`gapBetween…`, `highSpeedTrainEntersTunnelAtSpeed`
E-06	Very fast train (108 km/h)	Stations still recognised	Navigator	`veryFastTrain`
E-07	No GPS, no network underground	Speed model + station snapping	Navigator	all `RideSimulationTest`
E-08	No GPS, network available underground	Kalman correction, gating	Navigator	`networkLocationCorrects…`, `networkLocationUndergroundReducesError`
E-09	Wild network outlier	Rejected	Navigator	`wildNetworkOutlierIsRejected`
E-10	Boarding underground (no bearing)	Snap to station; direction from departure	Navigator	`boardingUndergroundWithoutGpsBearing`, `departureAcceleration…`
E-11	Train stops at a station	Snap, σ reset, cruise learnt	Navigator	`stopSnaps…`, `cruiseSpeedIsLearnt…`
E-12	Estimate reaches station before train	Held at platform until stop	Navigator	`estimateIsHeld…`

ID	Situation	Behaviour	Where	Test
E-13	Train skips a station	Released after allowance, event logged	Navigator	`skippedStationIsReleased…`
E-14	Train stops at a signal between stations	No snap; event logged	Navigator	—
E-15	Passenger walks on platform/in carriage	Train position unchanged	Navigator/SensorHub	`walkingDoesNotMoveTheTrain`
E-16	Interchange underground	Line chosen from departure bearing	Navigator	`interchangePicks…`
E-17	Direction unknown	Distance held until departure decides; *Flip*; destination	Navigator/UI	`departureAcceleration…`, `destinationSets…`
E-18	Strong compass disturbance (60°)	Still correct	Navigator	`strongMagneticDisturbance`
E-19	GPS returns after tunnel with poor accuracy	Needs confirmation	Navigator	`poorFixAfterOutageNeedsConfirmation`
E-20	GPS returns	Recovery error measured and logged	Navigator	`recoveryDriftIsMeasured…`
E-21	Single GPS jump	Rejected	Navigator	`implausibleGpsJumpIsRejected`
E-22	Wrong initial reference (3 consistent jumps)	New position accepted	Navigator	`repeatedConsistentJumps…`
E-23	Invalid fixes (0,0 / NaN / accuracy 0 / > 2 km)	Ignored	Navigator	`invalidFixesAreIgnored`
E-24	First location is network, never GPS	Adopted, lock to line if close	Navigator	`networkFixLocksOntoLine…`
E-25	No line stored, no GPS	DR with fading GPS speed; network blend	Navigator	`deadReckoningWithoutRoute`, `noLineUnderNetwork…`, `freeDeadReckoning…`
E-26	Cold start indoors (no satellites yet)	No invented movement	Navigator	`coldStartWithoutSatellites…` (found on device)
E-27	Uncertainty honesty	σ grows without GPS, resets at stops	Navigator	`uncertaintyGrows…`
E-28	Mock locations	Flagged in UI and log	Navigator/UI	`mockLocationsAreFlagged`

ID	Situation	Behaviour	Where	Test
E-29	Clock / time-zone change	No effect (monotonic time)	Everywhere	by design
E-30	Stale cached fix delivered late	Dropped (> 30 s)	SensorHub	by design
E-31	Sensor warm-up (< 10 samples)	Treated as unknown, not walking	SensorHub	found on device, fixed
E-32	No linear-acceleration sensor	Raw accelerometer − gravity	SensorHub	by design
E-33	No rotation-vector sensor	No world acceleration; warning; destination advised	SensorHub/UI	by design
E-34	No gyroscope	Stillness from acceleration only	SensorHub	by design
E-35	No barometer / step detector	Features skipped	SensorHub	by design
E-36	Device doze / long pause	dt capped at 1 s	SensorHub	by design
E-37	Screen off during trip	Foreground service + wake lock	Service	device
E-38	Battery optimisation	Warning + settings	UI	device
E-39	Location services off	Warning + enable dialog	UI (both apps)	device
E-40	Approximate location only	Warning; balanced fused updates	UI/SensorHub	by design
E-41	Notifications denied	Service runs; warning	UI	device (seen on test phone)
E-42	No Google Play services	Platform network provider	SensorHub	by design
E-43	No GNSS hardware	Fused ≤ 25 m trusted; warning	SensorHub/UI	by design
E-44	Overpass busy (HTTP 504) / timeout	Next server; back-off; lighter query	RouteService	seen on device, fixed
E-45	Captive portal / no real internet	Network WEAK; downloads may fail → back-off	NetworkStatus	by design

ID	Situation	Behaviour	Where	Test
E-46	Area already downloaded	No network call for 7 days	RouteService	device
E-47	Corrupt/partial cache	Deleted; atomic writes	RouteService	by design
E-48	Storage full / log unwritable	Fallback; error shown	TelemetryLogger	by design
E-49	App reinstalled (old public file not writable)	New MediaStore entry / fallback	TelemetryLogger	by design
E-50	Midnight	New daily log file	TelemetryLogger	by design
E-51	Process killed	Stored lines persist; GPS Logger reloads list	Controller/LocationLog	by design
E-52	Reversed / unordered OSM ways	Stitched correctly	TrackStitcher	`stitcherJoins…`
E-53	Disconnected OSM pieces	Not joined across > 50 m	TrackStitcher	`stitcherStopsAtLargeGaps`
E-54	Duplicate / far stations in OSM	Deduplicated / filtered	MetroRoute	`stationsAreSorted…`
E-55	End of line	Distance clamped; “End of line”	Navigator/UI	—
E-56	Standing at a snapped station with network	Network does not move it	Navigator	found in simulation, fixed

11. Accuracy results (simulation)

Simulation model: straight line with stations at 0, 1.1, 2.0, 3.6, 4.4, 6.2, 7.3 km; train accelerates/brakes at 1.0 m/s², dwells 25 s; IMU noise σ = 0.3 m/s² (0.5 in the disturbance case); compass error 25° (60° case); GPS 1 Hz ±4 m until 300 m then lost; loss reported 6 s later.

Scenario	Stations recognised	Error at stops	Max error while moving
80 km/h, enters tunnel at speed	B C D E F G (6/6)	0 m	323 m
108 km/h	6/6	0 m	424 m
Boarding underground, no GPS bearing	6/6	0 m	351 m
Compass error 60°, IMU noise 0.5	6/6	0 m	440 m
No network (seed 11)	6/6	0 m	352 m

Scenario	Stations recognised	Error at stops	Max error while moving
Network fix every 15 s (σ 80 m)	6/6	0 m	258 m
20 random seeds, compass 0–49°	6/6 in every run	—	—

Interpretation: station-level navigation (current/next station, stops remaining) is reliable in simulation; the metre-level position between stations is approximate, which is inherent to phone-sensor dead reckoning without external signals.

12. Testing

12.1 Automated tests (39, all passing)

MetroRouteTest (7): station sorting/dedup/filtering; projection distance and offset; pointAt ↔ projection round trip; next/previous station by direction; bearing by direction; stitching reversed + unordered ways; no stitching across large gaps.

UndergroundNavigatorTest (11): lock to track + direction from GPS bearing; advancing without GPS (constrained to track); stop snaps to nearest station ahead; hold at next station until stop; skipped station released; walking does not move the train; departure acceleration sets direction; destination sets direction and counts stops; dead reckoning without a line; recovery error measured; implausible jump rejected.

EdgeCaseTest (15): invalid fixes; network ignored while GPS good; network corrects track estimate; network outlier rejected; network-only lock to line; loss-gap dead reckoning; recovery confirmation; consistent jumps accepted; cruise learnt from station timing; interchange line selection; free-space network blend; uncertainty growth/reset; cold start without satellites; free DR uses last GPS speed; mock flag.

RideSimulationTest (6): the scenarios in §11 including 20 random seeds.

12.2 Static analysis

`./gradlew lintDebug`: 0 errors in all modules. Remaining warnings are cosmetic (newer library versions available, hard-coded UI strings, target SDK not latest).

12.3 On-device checks performed (Samsung Galaxy S24+, Android 14)

- Both apps install and run without crashes.
- Satellite fix accuracy 4 m, 33/35 satellites used; network status shown.
- Metro lines downloaded (6 Bengaluru Green/Yellow line routes, 32 stations on the Green line in correct order) and stored; offline reload after app restart; area memory prevents re-download.
- Warning banner detected disabled notifications.
- Issues found on device and fixed: Overpass 504 with the heavy query; line count shown late; sensor warm-up misclassified as walking; cold-start dead reckoning.

13. Field validation procedure (required before production)

1. Install Metro Navigator; grant all permissions; disable battery optimisation (*Fix* button).
2. Above ground near the line, online: wait for “Metro line data: saved…” (or press *Download whole city*).

3. Press **Start Logging**. Put the phone in a pocket for one ride and in the hand for another.
4. Ride at least 5 underground stations, including one boarding at an underground station and, if possible, an interchange. Note the true station and time of each stop.
5. After exiting, copy `Download/TestGPSLocation/Metro_Navigator_telemetry_<date>.txt`.
6. Check in the log: every "EVENT: Stopped at X" matches reality; "Departure detected" direction is correct; "GPS recovered … off by N m" is small; there are no "Train stopped between stations" events at real stations and no false stops while moving.

13.1 Tuning guide

Symptom in the log	Constant to adjust
Stops missed (train stood still but no "Stopped at")	Increase `STILL_ACCEL_STD` (e.g. 0.08) or `STILL_GYRO_MEAN`; decrease `STATION_STOP_MS` (e.g. 4000)
False stops while moving	Decrease `STILL_ACCEL_STD`; increase `STATION_STOP_MS`
Snapped to the wrong (next-next) station	Decrease `STOP_SNAP_AHEAD_FRACTION`
Station marked "passed" although the train stopped	Increase `OVERSHOOT_FRACTION`
Direction wrong after departure	Increase `DEPARTURE_MIN_ACCEL` or `DEPARTURE_WINDOW_SEC`
Estimate consistently behind/ahead between stations	Default cruise; learning happens automatically after the first segment
GPS "lost" too often above ground	Increase `GPS_TIMEOUT_MS`

14. Known limitations and future work

- Underground position between stations is approximate (hundreds of metres); station-level information is the reliable output.
- Direction relies on the compass at departure when neither GPS bearing nor destination is known; strongly magnetised carriages could exceed the ±90° tolerance (use *Flip* or choose a destination).
- Quality depends on OpenStreetMap data for the city.
- Possible enhancements: station stop-duration/time-table priors; Wi-Fi fingerprinting of stations; learning per-segment travel times across trips; map view; release build + Play Store listing; instrumented UI tests; upgrading AGP/Kotlin/libraries (lint lists newer versions).

15. Change history (from the original project)

Area	Change
Structure	4 modules (`app`, `core`, `trainbox`, `testerapp`) → 3 (`app`, `core`, `gpslogger`); dead duplicate classes removed
Correctness	Fixed: tunnel set-up never running; distance-to-next always wrong; history distance always 0; trainbox never logging (dt > 1 s); testerapp mixing simulated and real GPS; blank destination; `%n` in log lines; station-hold creeping; previous-station/segment-length bug

Area	Change
Platform	`POST_NOTIFICATIONS` declared; permission check before foreground start (Android 14 crash); foreground service type; MediaStore logging on Android 10+; wake lock
Performance	Motion processing throttled to 10 Hz; station distances precomputed; UI ≤ 4 Hz
New	Offline metro cache; track navigation; station hold/snap; departure/interchange detection; network Kalman corrections; uncertainty; GPS recovery confirmation; jump filter reset; cruise learning; back-off downloads; warning banner; GPS Logger app; 39 tests

16. Troubleshooting

Problem	Solution
“Searching for metro line…” forever	Needs one online moment within 2 km of the line; press *Refresh lines here* or *Download whole city*; check the status line for server errors
“No metro line within 400 m”	Normal when not at the metro; lines are saved
Direction shows “not confirmed”	Choose a destination or wait for the train to depart; use *Flip* if wrong
Logging stops with screen off	Press *Fix* on the battery-optimisation warning; keep the notification enabled
No log file in Download	See the log path on screen; if a write error is shown, free storage
GPS Logger list empty after reboot	It reloads today’s file on open; older days are in their own files